10 Python Case Studies

Flowcharts • Pseudocode • Python Source Code

Prepared for: First-Year B.Tech / Engineering Students

A practical problem-solving workbook with clear algorithms, visual flowcharts, runnable example programs, and sample outputs.

Important note: Some case studies do not specify exact thresholds, rates, or policies. The PDF uses clearly stated illustrative assumptions so the programs can be demonstrated. Replace them with your college or assignment rules if different.

How to Use This Workbook

For every case study, read the problem statement and assumptions first. Follow the flowchart to understand the order of operations, read the pseudocode as language-independent logic, and then run the Python source code. The sample output is based on the sample data included in that code.

General programming ideas used

• Input validation prevents invalid data from silently affecting results.
• Lists and dictionaries store multiple records and rules.
• Loops process applicants, passengers, expenses, transactions, or seats.
• Conditional statements apply eligibility rules and constraints.
• Sorting, counting, and aggregation create summaries and rankings.

Contents

1. University Scholarship Allocation

2. Railway Fare and Reservation Calculator

3. Multi-Currency Travel Budget Analysis

4. ATM Transaction Sequence Simulation

5. Student Performance Trend Analysis

6. University Admission Decision Engine

7. Student Timetable Conflict Detection

8. Nested List Classroom Occupancy Analysis

9. Food Waste Monitoring System

10. Campus Resource Allocation Under Constraints

CASE STUDY NOTES

1. University Scholarship Allocation

Problem Statement

This program checks student scholarship applications using aggregate marks, subject marks, attendance, and annual family income. It separates eligible and ineligible students and ranks eligible students according to the given rules.

Objectives

- To check whether each student satisfies all scholarship eligibility conditions.
- To identify and display the reason when a student is not eligible.
- To rank eligible students using aggregate marks, attendance, family income, and name.
- To practise lists, dictionaries, conditions, loops, and sorting in Python.

Assumptions

- Each applicant has a name, aggregate percentage, subject marks, attendance percentage, and annual family income.
- Minimum aggregate is 75%, every subject mark must be at least 60%, and attendance must be at least 75%.
- Annual family income must not exceed ■300,000.
- Ranking is by higher aggregate, then higher attendance, then lower income, and finally alphabetical name.
- The sample applicants and thresholds are for practice and can be changed to match actual scholarship rules.

These notes are written to match the Python code and sample data in the original workbook. Existing pages and code have been kept unchanged.

1. University Scholarship Allocation

Problem statement

Evaluate applicants using aggregate marks, subject-wise minimum scores, attendance, and family-income criteria. Rank eligible applicants and resolve ties using predefined rules.

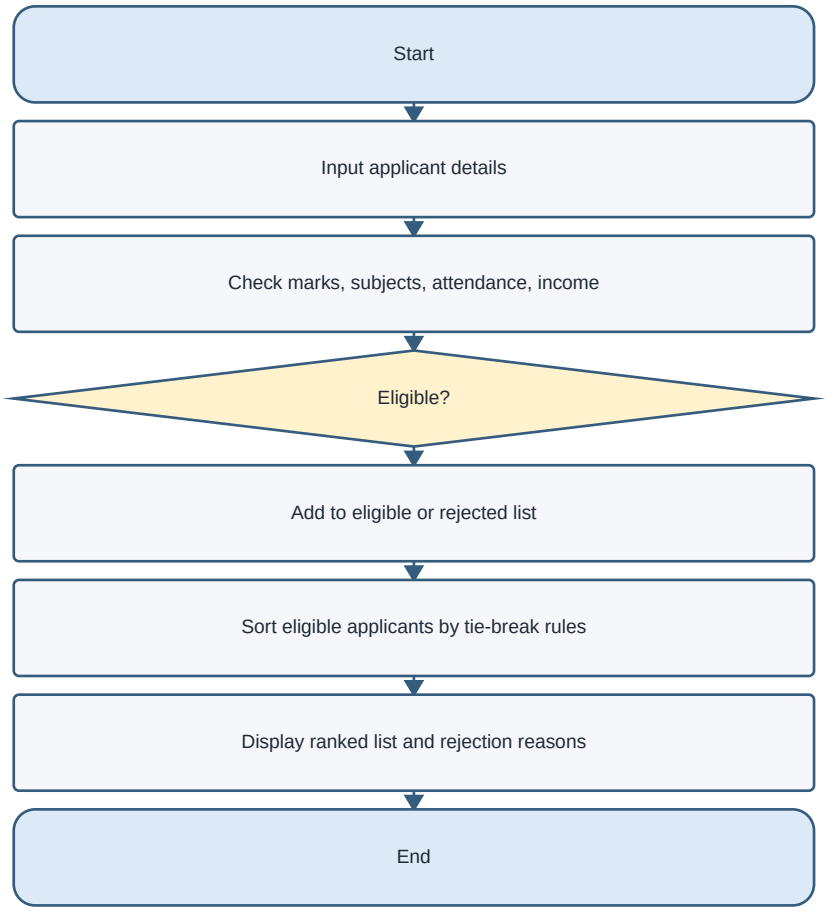
Assumptions and rules

• Each applicant has aggregate percentage, subject scores, attendance percentage, and annual family income.

• Illustrative eligibility rules: aggregate >= 75%, every required subject >= 60%, attendance >= 75%, and annual income <= ■300,000.

• Ranking order: higher aggregate, then higher attendance, then lower family income, then applicant name alphabetically. Change thresholds to match the university policy.

Flowchart



Pseudocode

```
BEGIN
  SET eligibility thresholds
  INPUT number of applicants
  FOR each applicant
    INPUT name, aggregate, subject scores, attendance, income
    IF aggregate, every subject, attendance, and income satisfy rules
      MARK applicant eligible
      ADD applicant to eligible list
```

```
    ELSE
      MARK applicant ineligible and record reason(s)
    ENDIF
  ENDFOR
  SORT eligible list by:
    aggregate descending, attendance descending,
    income ascending, name ascending
  DISPLAY eligible applicants with rank
  DISPLAY ineligible applicants with reason(s)
END
```

1. University Scholarship Allocation — Python Program

Python source code

```
# University Scholarship Allocation
MIN_AGGREGATE = 75
MIN_SUBJECT = 60
MIN_ATTENDANCE = 75
MAX_INCOME = 300000

applicants = [
    {"name": "Asha", "aggregate": 88, "subjects": [82, 91, 85],
     "attendance": 92, "income": 180000},
    {"name": "Bharat", "aggregate": 78, "subjects": [65, 72, 60],
     "attendance": 86, "income": 250000},
    {"name": "Charan", "aggregate": 74, "subjects": [80, 78, 75],
     "attendance": 95, "income": 150000},
    {"name": "Divya", "aggregate": 88, "subjects": [70, 84, 79],
     "attendance": 92, "income": 160000},
]

eligible, rejected = [], []
for student in applicants:
    reasons = []
    if student["aggregate"] < MIN_AGGREGATE:
        reasons.append("aggregate below minimum")
    if not student["subjects"] or min(student["subjects"]) < MIN_SUBJECT:
        reasons.append("subject score below minimum")
    if student["attendance"] < MIN_ATTENDANCE:
        reasons.append("attendance below minimum")
    if student["income"] > MAX_INCOME:
        reasons.append("income above limit")
    if reasons:
        rejected.append((student["name"], reasons))
    else:
        eligible.append(student)

eligible.sort(key=lambda s: (
    -s["aggregate"], -s["attendance"], s["income"], s["name"].lower()
))
print("SCHOLARSHIP RANK LIST")
for rank, student in enumerate(eligible, start=1):
    print(rank, student["name"], student["aggregate"],
          student["attendance"], student["income"])
print("\nNOT ELIGIBLE")
for name, reasons in rejected:
    print(name + ":", ", ".join(reasons))
```

Sample output

```
SCHOLARSHIP RANK LIST
1 Divya 88 92 160000
2 Asha 88 92 180000
3 Bharat 78 86 250000

NOT ELIGIBLE
Charan: aggregate below minimum
```

Understanding the solution

The program checks every eligibility condition and records rejection reasons. Sorting uses a tuple key: higher marks and attendance rank first, then lower income and alphabetical name break ties.

CASE STUDY NOTES

2. Railway Fare and Reservation Calculator

Problem Statement

This program calculates ticket fares for several passengers using travel distance, travel class, age concessions, reservation fees, and cancellation rules. It also displays payable fares, cancellation penalties, and refunds.

Objectives

- To calculate a passenger's base fare using distance and travel class.
- To apply the example age concessions and distance surcharge correctly.
- To calculate the reservation fee, cancellation penalty, and refund.
- To display totals for active tickets, penalties, and refunds.

Assumptions

- Example rates per kilometre are Sleeper ■1.20, AC 3 Tier ■2.00, and AC 2 Tier ■2.80.
- A ■50 surcharge is added when distance is greater than 500 km, and the reservation fee is ■40 per passenger.
- Children under 5 pay no base fare; ages 5–11 receive a 50% base-fare concession; passengers aged 60 or above receive a 10% base-fare concession.
- Concessions apply to the base fare only. The reservation fee is still added.
- Cancellation penalty is 25% of the calculated fare. These values are examples, not official railway tariff rules.
- Passenger data is supplied in the program as sample records.

These notes are written to match the Python code and sample data in the original workbook. Existing pages and code have been kept unchanged.

2. Railway Fare and Reservation Calculator

Problem statement

Calculate fares for multiple passengers using distance slabs, travel classes, age concessions, reservation fees, and cancellation penalties.

Assumptions and rules

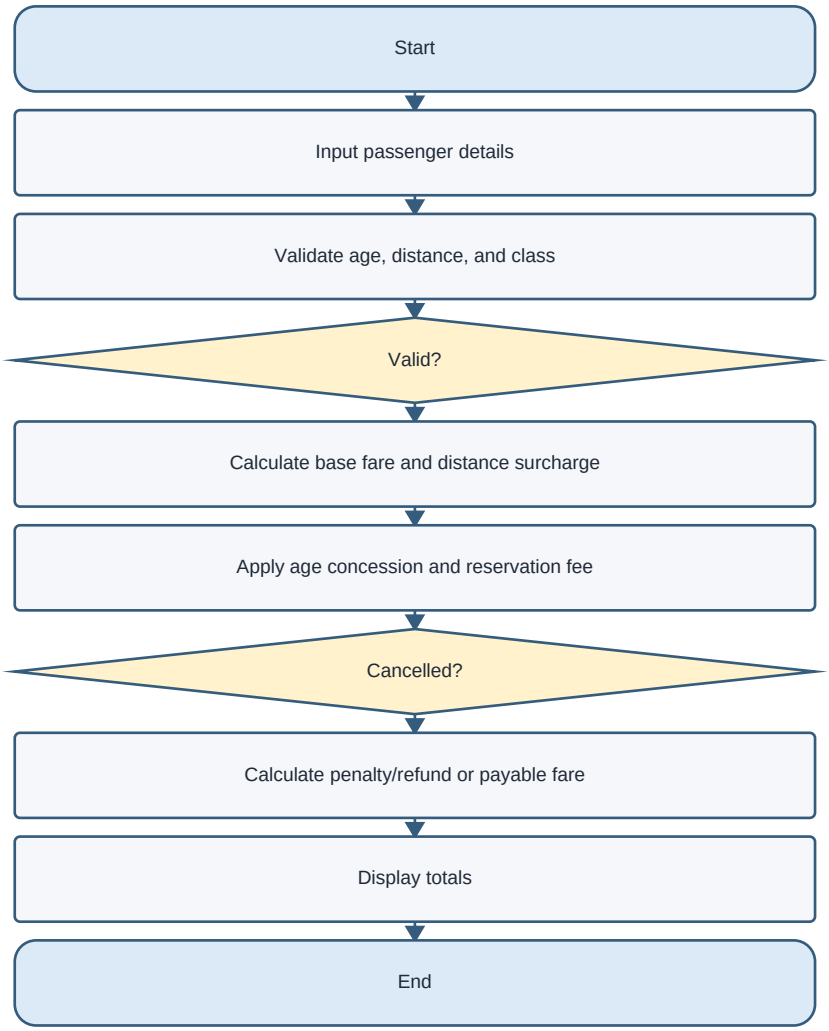
• Illustrative base rates per kilometre: Sleeper ■1.20, AC 3 Tier ■2.00, AC 2 Tier ■2.80.

• Distance slab surcharge: add ■50 for distances over 500 km. Reservation fee is ■40 per passenger.

• Children under 5 travel free; ages 5–11 receive 50% base-fare concession; ages 60+ receive 10% base-fare concession. Concessions apply only to base fare, not fees.

• Cancellation penalty is 25% of the final ticket fare. These are practice values, not actual railway tariff rules.

Flowchart



Pseudocode

```
BEGIN
  SET class rates, slab rule, fees, concessions
  INPUT passenger count
  FOR each passenger
    INPUT name, age, distance, class, cancellation choice
    VALIDATE age, distance, and class
    CALCULATE base fare = distance * class rate
```

```
    APPLY distance surcharge when applicable
    APPLY age concession to base fare
    IF age < 5, set base fare to zero
    ADD reservation fee
    IF cancelled, calculate cancellation penalty and refund
    DISPLAY fare details
  ENDFOR
  DISPLAY total payable and total refund/penalty
END
```

2. Railway Fare and Reservation Calculator — Python Program

Python source code

```
# Railway Fare and Reservation Calculator
rates = {"sleeper": 1.20, "ac3": 2.00, "ac2": 2.80}
RESERVATION_FEE = 40
SLAB_LIMIT = 500
SLAB_SURCHARGE = 50
CANCELLATION_RATE = 0.25

passengers = [
    {"name": "Ravi", "age": 25, "distance": 420, "class": "sleeper",
     "cancel": False},
    {"name": "Meena", "age": 65, "distance": 620, "class": "ac3",
     "cancel": True},
    {"name": "Kiran", "age": 8, "distance": 300, "class": "sleeper",
     "cancel": False},
]

total_payable = total_refund = total_penalty = 0
for p in passengers:
    cls = p["class"].lower()
    if p["age"] < 0 or p["distance"] <= 0 or cls not in rates:
        print(p["name"], "INVALID passenger details")
        continue

    base = p["distance"] * rates[cls]
    if p["distance"] > SLAB_LIMIT:
        base += SLAB_SURCHARGE

    if p["age"] < 5:
        base = 0
    elif 5 <= p["age"] <= 11:
        base *= 0.50
    elif p["age"] >= 60:
        base *= 0.90

    fare = round(base + RESERVATION_FEE, 2)
    if p["cancel"]:
        penalty = round(fare * CANCELLATION_RATE, 2)
        refund = round(fare - penalty, 2)
        total_penalty += penalty
        total_refund += refund
        print(p["name"], "Fare:", fare, "Penalty:", penalty,
              "Refund:", refund)
    else:
        total_payable += fare
        print(p["name"], "Fare payable:", fare)

print("Total payable for active tickets:", round(total_payable, 2))
print("Total cancellation penalties:", round(total_penalty, 2))
print("Total refunds:", round(total_refund, 2))
```

Sample output

```
Ravi Fare payable: 544.0
Meena Fare: 1201.0 Penalty: 300.25 Refund: 900.75
Kiran Fare payable: 220.0
Total payable for active tickets: 764.0
Total cancellation penalties: 300.25
Total refunds: 900.75
```

Understanding the solution

The program calculates base fare from distance and class, applies the illustrative age rule, adds the reservation fee, and separates active-ticket payments from cancellation refunds and penalties.

CASE STUDY NOTES

3. Multi-Currency Travel Budget Analysis

Problem Statement

This program converts travel expenses from different currencies into Indian rupees, adds a conversion fee, totals expenses by category, finds the largest spending category, and checks whether spending is within budget.

Objectives

- To convert foreign-currency expenses into INR using supplied exchange rates.
- To calculate and add a conversion fee to each valid expense.
- To calculate category totals and total travel spending.
- To identify the largest expense category and compare total spending with the budget.

Assumptions

- All expenses are converted into Indian rupees (INR).
- The sample exchange rates are USD = ■83.00, EUR = ■90.00, and JPY = ■0.56 per one unit of currency.
- A 2% conversion fee is applied to each converted expense.
- The sample budget is ■50,000.
- Negative expense amounts and currencies without a supplied rate are skipped.
- Exchange rates and sample amounts are illustrative and may not reflect live market rates.

These notes are written to match the Python code and sample data in the original workbook. Existing pages and code have been kept unchanged.

3. Multi-Currency Travel Budget Analysis

Problem statement

Convert expenses using supplied exchange rates, include conversion fees, compare total spending with a budget, and identify the largest expense category.

Assumptions and rules

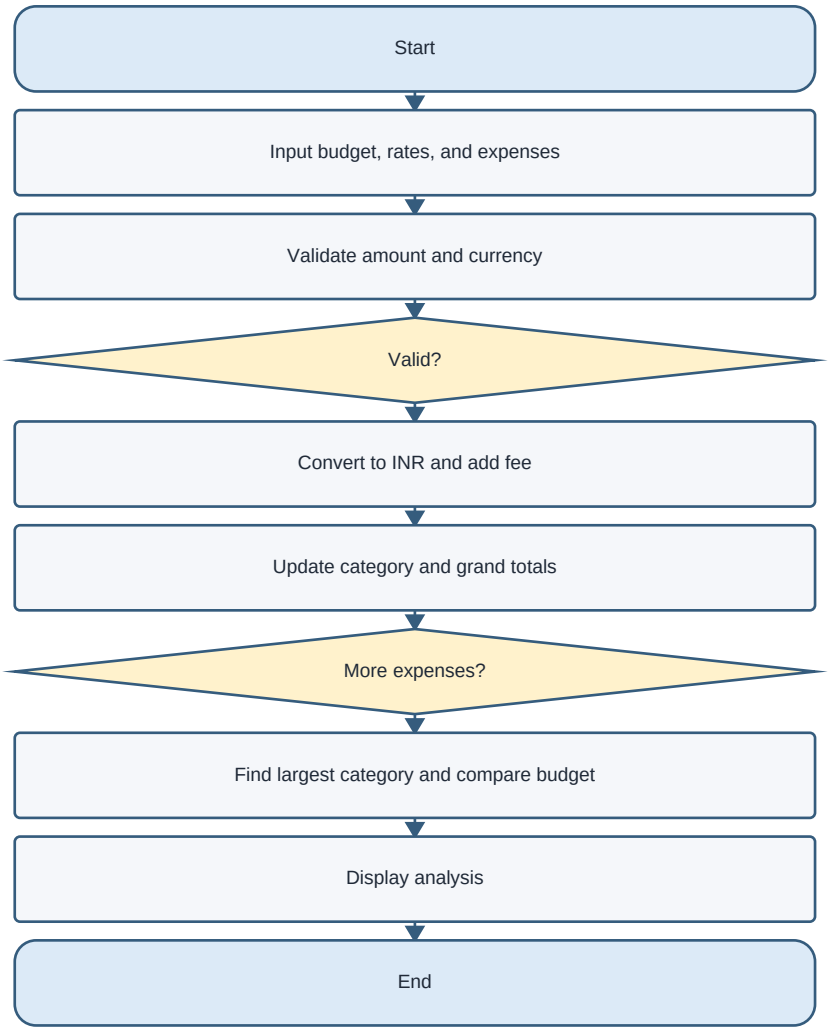
• All expenses are entered in a foreign currency and converted to Indian rupees (INR).

• Exchange rates mean INR received/required for one unit of the foreign currency.

• A conversion fee of 2% is applied to each converted expense.

• Budget and category totals are compared in INR; sample values are illustrative.

Flowchart



Pseudocode

```
BEGIN
  INPUT budget in INR and conversion fee rate
  INPUT exchange rate for each currency
  INPUT expense records: category, amount, currency
  SET total spending to zero
  FOR each expense
    IF currency rate is missing or amount is negative
      REPORT invalid expense
    ELSE
```

```
      CONVERT amount to INR
      CALCULATE conversion fee
      ADD converted amount and fee to category and total
    ENDIF
  ENDFOR
  FIND category with maximum total
  COMPARE total spending with budget
  DISPLAY category totals, largest category, and budget status
END
```

3. Multi-Currency Travel Budget Analysis — Python Program

Python source code

```
# Multi-Currency Travel Budget Analysis
budget_inr = 50000
fee_rate = 0.02
rates_to_inr = {"USD": 83.0, "EUR": 90.0, "JPY": 0.56}
expenses = [
    ("Hotel", 200, "USD"),
    ("Food", 80, "EUR"),
    ("Transport", 5000, "JPY"),
    ("Shopping", 100, "USD"),
    ("Food", 30, "USD"),
]

category_totals = {}
total_spending = 0.0
for category, amount, currency in expenses:
    if amount < 0 or currency not in rates_to_inr:
        print("Skipped invalid expense:", category, amount, currency)
        continue
    converted = amount * rates_to_inr[currency]
    fee = converted * fee_rate
    total = converted + fee
    category_totals[category] = category_totals.get(category, 0) + total
    total_spending += total

print("CATEGORY TOTALS (INR)")
for category, amount in sorted(category_totals.items()):
    print(category, round(amount, 2))
if category_totals:
    largest = max(category_totals, key=category_totals.get)
    print("Largest expense category:", largest)
print("Total spending:", round(total_spending, 2))
print("Budget:", budget_inr)
if total_spending > budget_inr:
    print("Over budget by", round(total_spending - budget_inr, 2), "INR")
else:
    print("Within budget; remaining:", round(budget_inr - total_spending, 2),
          "INR")
```

Sample output

```
CATEGORY TOTALS (INR)
Food 9843.0
Hotel 16932.0
Shopping 8466.0
Transport 2856.0
Largest expense category: Hotel
Total spending: 38097.0
Budget: 50000
Within budget; remaining: 11903.0 INR
```

Understanding the solution

Each expense is converted with its currency rate, then a percentage fee is added. Category totals are accumulated in a dictionary to find the largest category and compare spending with the budget.

CASE STUDY NOTES

4. ATM Transaction Sequence Simulation

Problem Statement

This program simulates a simple ATM by processing deposits and withdrawals, updating the account balance, rejecting invalid transactions, and showing a final transaction summary.

Objectives

- To update the account balance when a valid deposit or withdrawal is made.
- To reject unsupported transaction types and non-positive amounts.
- To prevent withdrawals that exceed the per-transaction limit or available balance.
- To display accepted and rejected transaction counts, totals, and final balance.

Assumptions

- The opening balance is ■5,000.
- The maximum withdrawal allowed in one transaction is ■2,000.
- Transactions are supplied as type-and-amount pairs in the program.
- Only deposit and withdraw transaction types are accepted, and each amount must be positive.
- A withdrawal is accepted only when it does not exceed either the withdrawal limit or the available balance.
- This is a classroom simulation and does not connect to a real bank account.

These notes are written to match the Python code and sample data in the original workbook. Existing pages and code have been kept unchanged.

4. ATM Transaction Sequence Simulation

Problem statement

Process deposits and withdrawals, maintain a simulated balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

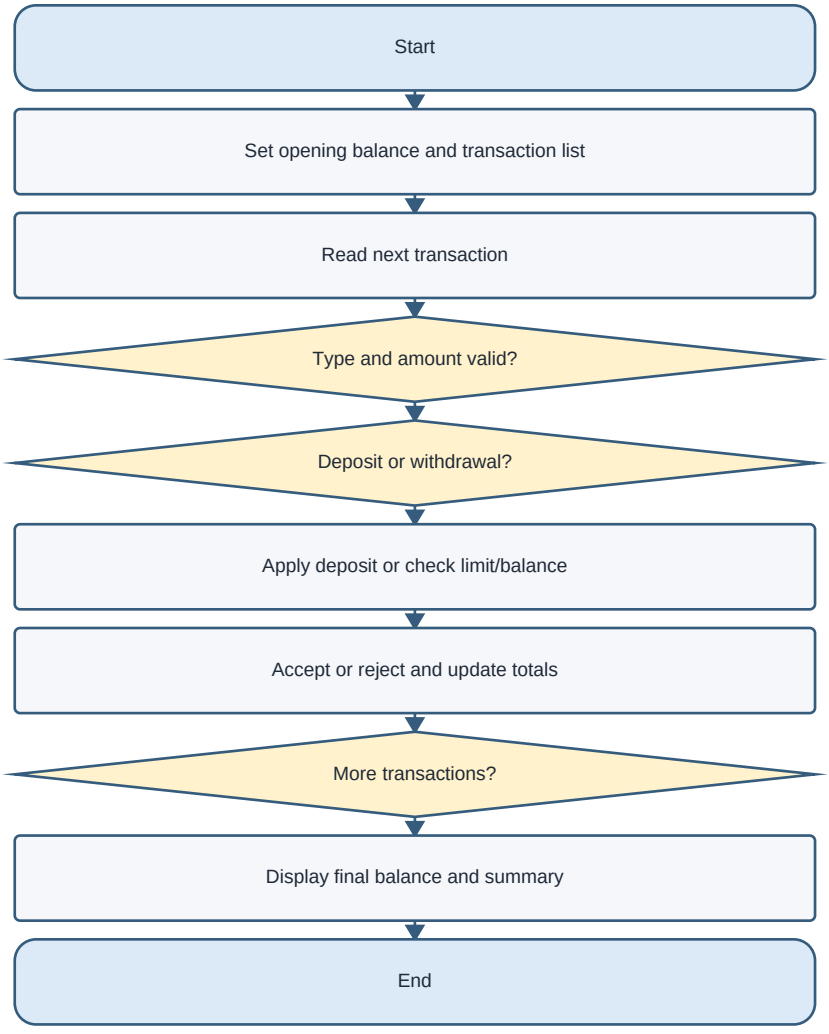
Assumptions and rules

• Opening balance is ■5,000 and maximum withdrawal per transaction is ■2,000.

• Transactions are supplied as type and amount pairs. Only positive amounts and known transaction types are accepted.

• Withdrawals cannot exceed the available balance. This is a simulation and does not connect to a real bank.

Flowchart



Pseudocode

```
BEGIN
  INPUT opening balance and transaction sequence
  SET deposit, withdrawal, accepted, rejected counters to zero
  FOR each transaction
    VALIDATE type and positive amount
    IF invalid, reject transaction
    ELSE IF deposit, add amount to balance and deposit total
    ELSE IF withdrawal exceeds limit, reject transaction
    ELSE IF withdrawal exceeds balance, reject transaction
    ELSE subtract amount and update withdrawal total
```

```
    ENDIF
  ENDFOR
  DISPLAY final balance and transaction summary
END
```

4. ATM Transaction Sequence Simulation — Python Program

Python source code

```
# ATM Transaction Sequence Simulation
balance = 5000.00
withdrawal_limit = 2000.00
transactions = [
    ("deposit", 1500),
    ("withdraw", 1200),
    ("withdraw", 2500),
    ("withdraw", 6000),
    ("withdraw", 500),
    ("deposit", -100),
    ("transfer", 100),
]

deposit_total = withdrawal_total = 0.0
accepted = rejected = 0
for kind, amount in transactions:
    kind = kind.lower()
    if amount <= 0 or kind not in ("deposit", "withdraw"):
        print("REJECTED:", kind, amount, "- invalid transaction")
        rejected += 1
    elif kind == "deposit":
        balance += amount
        deposit_total += amount
        accepted += 1
        print("ACCEPTED deposit:", amount, "Balance:", round(balance, 2))
    elif amount > withdrawal_limit:
        print("REJECTED withdrawal:", amount, "- exceeds per-transaction limit")
        rejected += 1
    elif amount > balance:
        print("REJECTED withdrawal:", amount, "- insufficient balance")
        rejected += 1
    else:
        balance -= amount
        withdrawal_total += amount
        accepted += 1
        print("ACCEPTED withdrawal:", amount, "Balance:", round(balance, 2))

print("\nTRANSACTION SUMMARY")
print("Opening balance: 5000.00")
print("Deposits:", round(deposit_total, 2))
print("Withdrawals:", round(withdrawal_total, 2))
print("Accepted:", accepted, "Rejected:", rejected)
print("Final balance:", round(balance, 2))
```

Sample output

```
ACCEPTED deposit: 1500 Balance: 6500.0
ACCEPTED withdrawal: 1200 Balance: 5300.0
REJECTED withdrawal: 2500 - exceeds per-transaction limit
REJECTED withdrawal: 6000 - exceeds per-transaction limit
ACCEPTED withdrawal: 500 Balance: 4800.0
REJECTED: deposit -100 - invalid transaction
REJECTED: transfer 100 - invalid transaction

TRANSACTION SUMMARY
Opening balance: 5000.00
Deposits: 1500.0
Withdrawals: 1700.0
Accepted: 3 Rejected: 4
Final balance: 4800.0
```

Understanding the solution

Each transaction is validated before the balance changes. A rejected transaction does not affect the balance or accepted deposit/withdrawal totals.

CASE STUDY NOTES

5. Student Performance Trend Analysis

Problem Statement

This program compares student marks across examinations, identifies improvement or decline in each subject, finds consistently weak subjects, calculates the latest average, and assigns a performance category.

Objectives

- To compare the first and latest marks for each subject.
- To label a subject trend as improved, declined, or no change.
- To identify subjects in which all recorded marks are below 50%.
- To calculate the latest overall average and classify student performance.

Assumptions

- Each subject should have at least two exam scores.
- All marks are percentages from 0 to 100.
- A subject is considered consistently weak when every recorded score is below 50%.
- Classification uses the latest overall average: Excellent ≥ 85, Good ≥ 70, Needs Improvement ≥ 50, and At Risk < 50.
- The sample student scores are fixed example data in the program.
- Invalid subject score data is reported and not used in that subject's trend calculation.

These notes are written to match the Python code and sample data in the original workbook. Existing pages and code have been kept unchanged.

5. Student Performance Trend Analysis

Problem statement

Compare assessment scores across examinations, calculate improvements and declines, identify consistently weak subjects, and classify students using defined criteria.

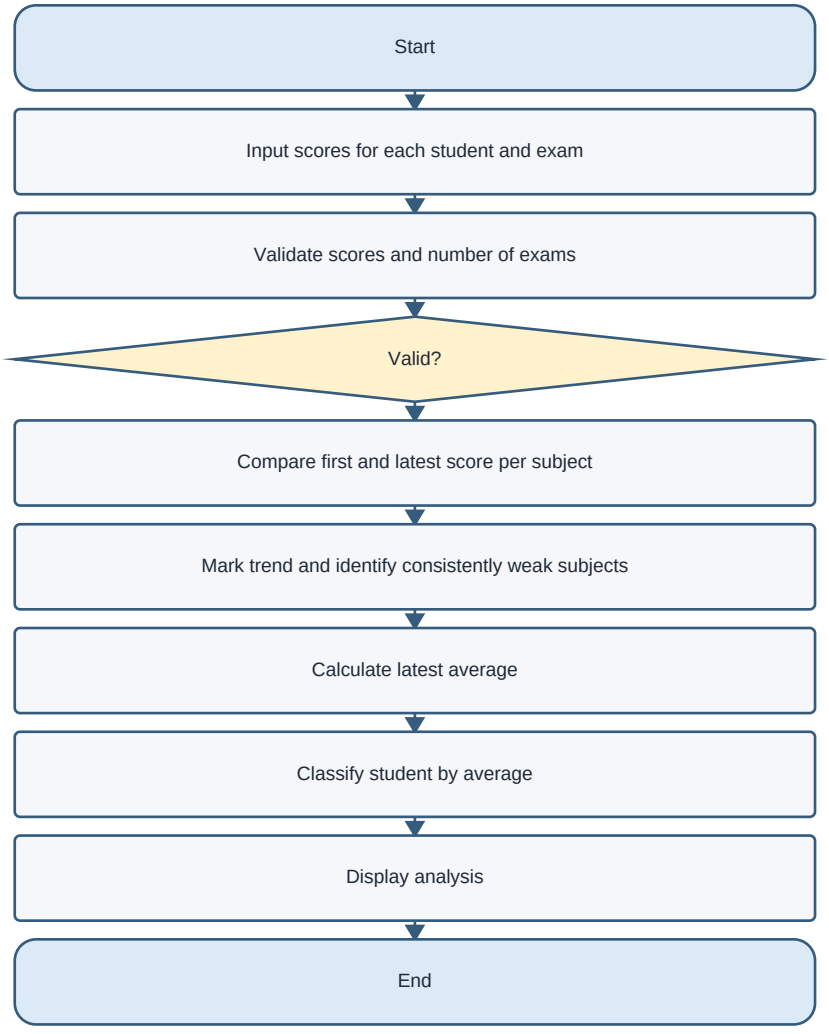
Assumptions and rules

• Scores are percentages from 0 to 100, with at least two exam scores per subject.

• A subject is consistently weak if every exam score for that subject is below 50%.

• Classification uses the latest overall average: Excellent >= 85, Good >= 70, Needs Improvement >= 50, At Risk < 50.

Flowchart



Pseudocode

```
BEGIN
  INPUT students and subject scores for each exam
  FOR each student
    VALIDATE every score is between 0 and 100
    FOR each subject
      CALCULATE change = latest score - first score
      LABEL improvement, decline, or no change
      IF every score is below weak-subject threshold
        ADD subject to weak-subject list
      ENDIF
```

```
    ENDFOR
    CALCULATE latest overall average
    CLASSIFY student using defined ranges
    DISPLAY trends, weak subjects, average, and class
  ENDFOR
END
```

5. Student Performance Trend Analysis — Python Program

Python source code

```
# Student Performance Trend Analysis
students = {
    "Asha": {
        "Maths": [62, 70, 82],
        "Physics": [48, 46, 49],
        "Programming": [70, 78, 88],
    },
    "Balu": {
        "Maths": [80, 75, 72],
        "Physics": [65, 68, 70],
        "Programming": [55, 61, 66],
    },
}

for name, subjects in students.items():
    latest_scores = []
    weak_subjects = []
    print("\nStudent:", name)
    for subject, scores in subjects.items():
        if len(scores) < 2 or any(s < 0 or s > 100 for s in scores):
            print(subject, "- invalid score data")
            continue
        change = scores[-1] - scores[0]
        if change > 0:
            trend = "Improved"
        elif change < 0:
            trend = "Declined"
        else:
            trend = "No change"
        if all(score < 50 for score in scores):
            weak_subjects.append(subject)
        latest_scores.append(scores[-1])
        print(subject, "change:", change, "points -", trend)

    average = sum(latest_scores) / len(latest_scores) if latest_scores else 0
    if average >= 85:
        category = "Excellent"
    elif average >= 70:
        category = "Good"
    elif average >= 50:
        category = "Needs Improvement"
    else:
        category = "At Risk"
    print("Latest average:", round(average, 2))
    print("Classification:", category)
    print("Consistently weak subjects:", weak_subjects or "None")
```

Sample output

```
Student: Asha
Maths change: 20 points - Improved
Physics change: 1 points - Improved
Programming change: 18 points - Improved
Latest average: 73.0
Classification: Good
Consistently weak subjects: ['Physics']

Student: Balu
Maths change: -8 points - Declined
Physics change: 5 points - Improved
Programming change: 11 points - Improved
Latest average: 69.33
Classification: Needs Improvement
Consistently weak subjects: None
```

Understanding the solution

The first and latest scores show the overall change for each subject. The program checks consistently low subjects and classifies the student using the latest average.

CASE STUDY NOTES

6. University Admission Decision Engine

Problem Statement

This program checks applicants against course-specific admission criteria, records reasons for rejection, and ranks eligible applicants separately for each course.

Objectives

- To store admission requirements for different courses.
- To check aggregate marks and the required subject mark for the selected course.
- To record applicants who do not satisfy the required criteria.
- To rank eligible applicants using the relevant subject mark, aggregate, and alphabetical name.

Assumptions

- Each applicant selects one course and provides aggregate, Mathematics, Physics, and Chemistry marks.
- CSE requires aggregate ≥ 80 and Mathematics ≥ 75.
- ECE requires aggregate ≥ 75 and Physics ≥ 70.
- Mechanical requires aggregate ≥ 70 and Mathematics ≥ 60.
- Eligible applicants are ranked by higher relevant subject score, then higher aggregate, then alphabetical name.
- These are illustrative admission rules for the program, not official college admission criteria.

These notes are written to match the Python code and sample data in the original workbook. Existing pages and code have been kept unchanged.

6. University Admission Decision Engine

Problem statement

Evaluate candidates using subject eligibility, aggregate marks, course-specific thresholds, and tie-breaking criteria, then rank eligible applicants.

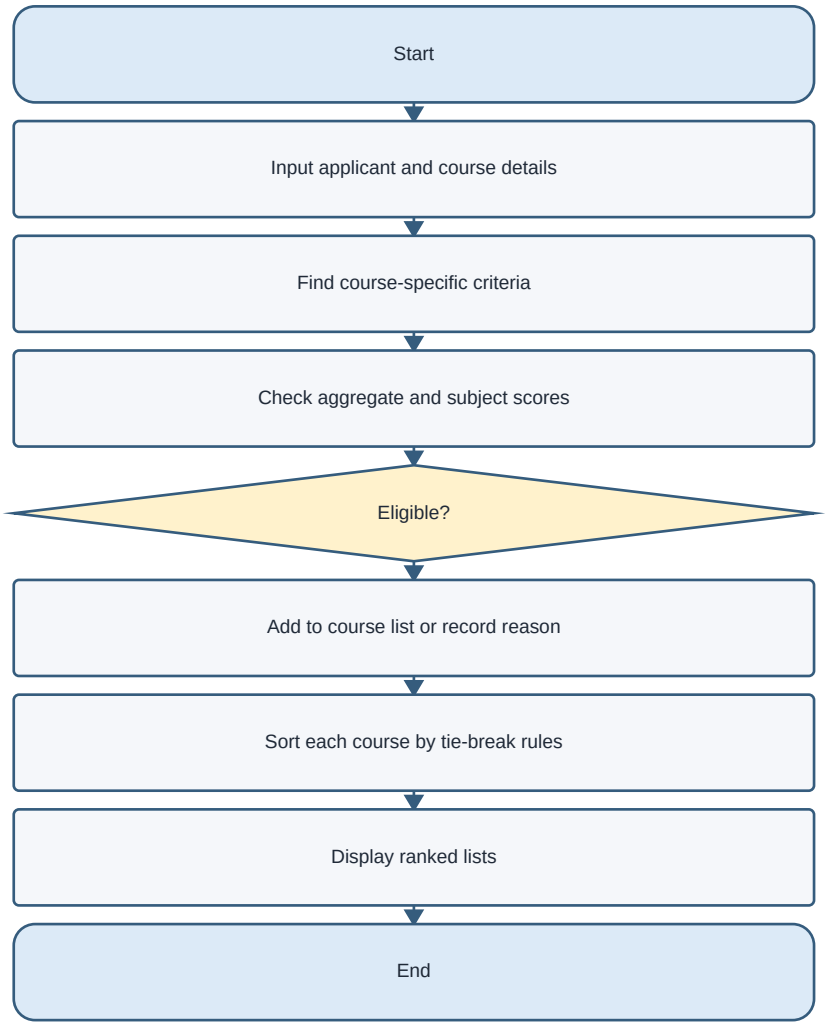
Assumptions and rules

• Applicants apply to one course and provide aggregate percentage plus Mathematics, Physics, and Chemistry marks.

• Illustrative course rules: CSE requires aggregate >= 80 and Maths >= 75; ECE requires aggregate >= 75 and Physics >= 70; Mechanical requires aggregate >= 70 and Maths >= 60.

• Ties are resolved by higher relevant subject score, then higher aggregate, then alphabetical name. These are example rules only.

Flowchart



Pseudocode

```
BEGIN
  DEFINE eligibility rules for each course
  INPUT applicant records
  FOR each applicant
    FIND the rule for the selected course
    CHECK aggregate and required subject threshold(s)
    IF all course rules pass, add applicant to eligible list
    ELSE record reason for ineligibility
```

```
  ENDFOR
  FOR each course
    SORT eligible applicants using course tie-break rules
    ASSIGN ranks
    DISPLAY ranked applicants
  ENDFOR
END
```

6. University Admission Decision Engine — Python Program

Python source code

```
# University Admission Decision Engine
rules = {
    "CSE": {"aggregate": 80, "subject": "Maths", "minimum": 75},
    "ECE": {"aggregate": 75, "subject": "Physics", "minimum": 70},
    "Mechanical": {"aggregate": 70, "subject": "Maths", "minimum": 60},
}
applicants = [
    {"name": "Asha", "course": "CSE", "aggregate": 88,
     "Maths": 90, "Physics": 82, "Chemistry": 85},
    {"name": "Bharat", "course": "CSE", "aggregate": 84,
     "Maths": 92, "Physics": 80, "Chemistry": 79},
    {"name": "Charan", "course": "ECE", "aggregate": 77,
     "Maths": 72, "Physics": 74, "Chemistry": 76},
    {"name": "Divya", "course": "Mechanical", "aggregate": 73,
     "Maths": 68, "Physics": 65, "Chemistry": 70},
]
by_course = {course: [] for course in rules}
rejected = []

for person in applicants:
    course = person["course"]
    if course not in rules:
        rejected.append((person["name"], "unknown course"))
        continue
    rule = rules[course]
    reasons = []
    if person["aggregate"] < rule["aggregate"]:
        reasons.append("aggregate below course threshold")
    if person[rule["subject"]] < rule["minimum"]:
        reasons.append(rule["subject"] + " below course threshold")
    if reasons:
        rejected.append((person["name"], "; ".join(reasons)))
    else:
        by_course[course].append(person)

for course, people in by_course.items():
    subject = rules[course]["subject"]
    people.sort(key=lambda p: (-p[subject], -p["aggregate"], p["name"].lower()))
    print("\n", course, "RANK LIST")
    for rank, p in enumerate(people, 1):
        print(rank, p["name"], "aggregate:", p["aggregate"],
              subject + ":", p[subject])
print("\nNOT ELIGIBLE")
for name, reason in rejected:
    print(name + ":", reason)
```

Sample output

```
CSE RANK LIST
1 Bharat aggregate: 84 Maths: 92
2 Asha aggregate: 88 Maths: 90

ECE RANK LIST

Mechanical RANK LIST
1 Divya aggregate: 73 Maths: 68

NOT ELIGIBLE
Charan: Physics below course threshold
```

Understanding the solution

Eligibility rules are stored by course, so each course can have different thresholds. Applicants are ranked within their selected course using the required subject score, aggregate, and name.

CASE STUDY NOTES

7. Student Timetable Conflict Detection

Problem Statement

This program checks a proposed class timetable for overlapping class times, rooms booked for more than one class at the same time, and rooms marked as unavailable.

Objectives

- To convert time values into a format that is easy to compare.
- To identify classes scheduled at overlapping times on the same day.
- To detect when the same room is double-booked during overlapping periods.
- To report classes assigned to unavailable rooms and decide whether the timetable is valid.

Assumptions

- Each timetable entry contains course, day, start time, end time, and room.
- Times use the 24-hour HH:MM format.
- Two classes overlap when they occur on the same day and the later start is earlier than the earlier end.
- Classes that meet exactly at an endpoint, such as 10:00–11:00 and 11:00–12:00, are not overlapping.
- Rooms listed in the unavailable-room set cannot be used.
- The program uses the sample timetable and room list included in the code.

These notes are written to match the Python code and sample data in the original workbook. Existing pages and code have been kept unchanged.

7. Student Timetable Conflict Detection

Problem statement

Compare course schedules represented by strings and tuples, detect overlapping periods, identify unavailable rooms, and validate a proposed timetable.

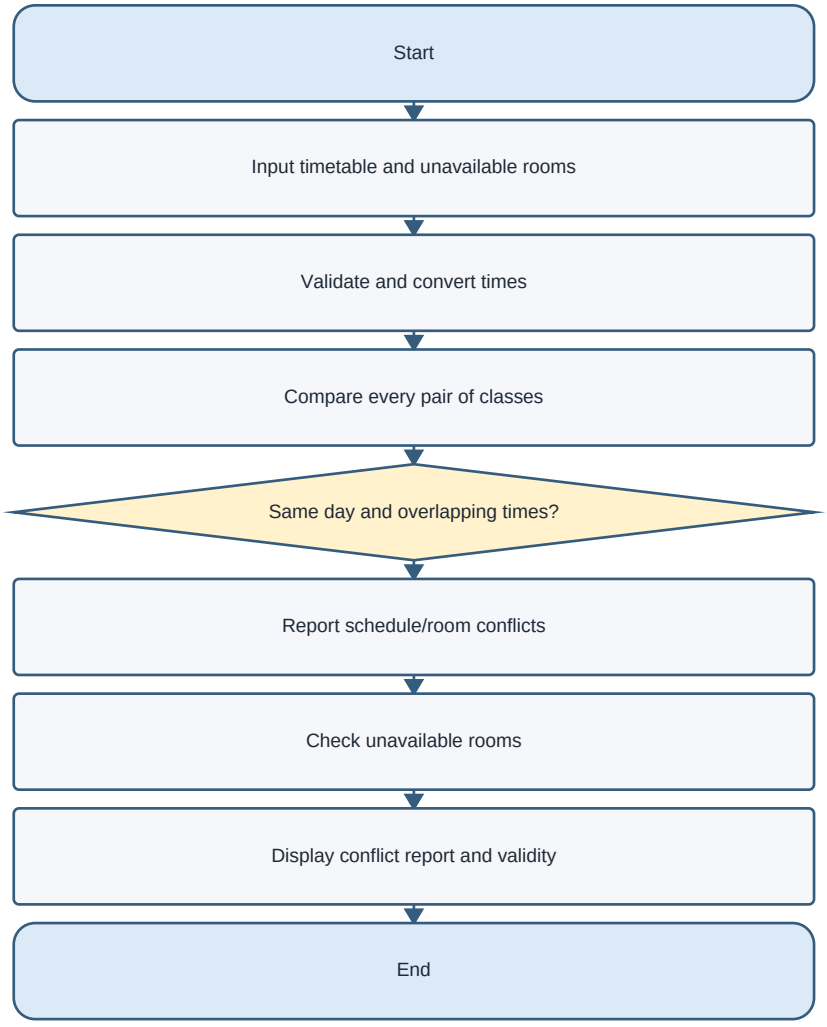
Assumptions and rules

• Each class is represented as (course, day, start_time, end_time, room), with times in 24-hour HH:MM format.

• Two classes conflict when they occur on the same day and their time intervals overlap; touching endpoints (for example, 10:00–11:00 and 11:00–12:00) are not considered overlapping.

• A room is unavailable if it appears in a supplied unavailable-room set.

Flowchart



Pseudocode

```
BEGIN
  INPUT timetable entries and unavailable rooms
  CONVERT time strings to minutes after midnight
  VALIDATE each entry's day, time range, and room
  FOR each pair of classes
    IF same day and intervals overlap
      REPORT schedule conflict
    ENDIF
    IF same room and intervals overlap
```

```
      REPORT room conflict
    ENDIF
  ENDFOR
  FOR each class
    IF room is unavailable, mark timetable invalid
  ENDFOR
  DISPLAY all conflicts and overall validation result
END
```

7. Student Timetable Conflict Detection — Python Program

Python source code

```
# Student Timetable Conflict Detection
# Entry: (course, day, start, end, room)
timetable = [
    ("Python", "Mon", "09:00", "10:00", "R101"),
    ("Maths", "Mon", "09:30", "10:30", "R102"),
    ("Physics", "Mon", "10:00", "11:00", "R101"),
    ("English", "Tue", "11:00", "12:00", "R103"),
]
unavailable_rooms = {"R103"}

def to_minutes(time_text):
    hour, minute = map(int, time_text.split(":"))
    if not (0 <= hour < 24 and 0 <= minute < 60):
        raise ValueError("Invalid time: " + time_text)
    return hour * 60 + minute

valid_entries = []
for entry in timetable:
    course, day, start, end, room = entry
    try:
        start_m, end_m = to_minutes(start), to_minutes(end)
        if start_m >= end_m:
            print("INVALID time interval:", course)
        else:
            valid_entries.append((course, day, start_m, end_m, room))
    except ValueError as error:
        print("INVALID entry:", error)

conflicts = []
for i in range(len(valid_entries)):
    c1, d1, s1, e1, r1 = valid_entries[i]
    if r1 in unavailable_rooms:
        conflicts.append(f"{c1}: room {r1} is unavailable")
    for j in range(i + 1, len(valid_entries)):
        c2, d2, s2, e2, r2 = valid_entries[j]
        overlap = d1 == d2 and max(s1, s2) < min(e1, e2)
        if overlap:
            conflicts.append(f"{c1} and {c2}: overlapping class times")
            if r1 == r2:
                conflicts.append(f"{c1} and {c2}: room {r1} double-booked")

print("CONFLICT REPORT")
if conflicts:
    for item in conflicts:
        print("-", item)
    print("Timetable valid:", False)
else:
    print("No conflicts found.")
    print("Timetable valid:", True)
```

Sample output

```
CONFLICT REPORT
- Python and Maths: overlapping class times
- English: room R103 is unavailable
Timetable valid: False
```

Understanding the solution

Times are converted into minutes, which makes overlap checking straightforward. Two intervals overlap when the later start is earlier than the earlier end. The program also checks room availability.

CASE STUDY NOTES

8. Nested List Classroom Occupancy Analysis

Problem Statement

This program represents classroom seats using nested lists and processes student seat requests. It prevents invalid allocations and provides a summary of occupied, reserved, and vacant seats.

Objectives

- To represent classrooms, rows, and seats using nested lists.
- To allocate a student only when the room and seat are valid and available.
- To reject reserved seats, already occupied seats, duplicate student IDs, and unknown rooms.
- To count occupied, reserved, and vacant seats for each classroom.

Assumptions

- Each classroom is represented as a list of rows, with each row containing seat entries.
- None means a vacant seat, a student ID means an occupied seat, and RESERVED means a seat that cannot be allocated.
- Seat row and seat indices are zero-based, as in Python lists.
- A student ID can be assigned to at most one seat across all classrooms.
- Only valid room and seat positions can be used.
- The sample classroom layouts and assignments are for demonstration.

These notes are written to match the Python code and sample data in the original workbook. Existing pages and code have been kept unchanged.

8. Nested List Classroom Occupancy Analysis

Problem statement

Use nested lists to represent classrooms and seats, allocate students, preserve reserved seats, detect duplicate assignments, and calculate vacant capacity.

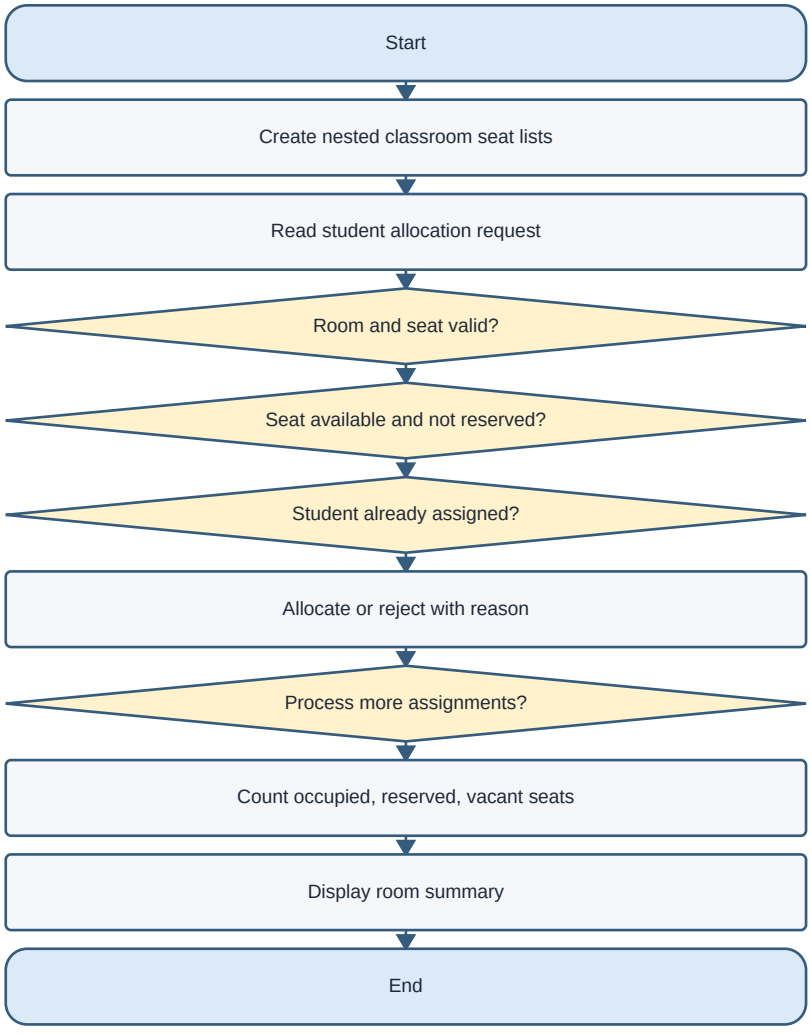
Assumptions and rules

• Each classroom is a nested list; each seat contains None (vacant), a student ID, or the string RESERVED.

• Seat indices are zero-based in Python.

• A student ID must appear at most once across all classrooms. Reserved seats cannot be allocated.

Flowchart



Pseudocode

```
BEGIN
  CREATE nested lists for rooms and seats
  MARK reserved seats
  INPUT student-to-room-and-seat assignments
  FOR each assignment
    CHECK room and seat indices
    CHECK whether seat is reserved or occupied
    CHECK whether student ID was already assigned
    IF all checks pass, allocate student
    ELSE report rejected assignment
```

```
  ENDFOR
  FOR each room
    COUNT total, occupied, reserved, and vacant seats
  ENDFOR
  DISPLAY classroom layouts and capacity summary
END
```

8. Nested List Classroom Occupancy Analysis — Python Program

Python source code

```
# Nested List Classroom Occupancy Analysis
# None = vacant; "RESERVED" = unavailable seat
classrooms = {
    "A": [[None, None, "RESERVED"], [None, None, None]],
    "B": [[None, "RESERVED"], [None, None]],
}
assignments = [
    ("ST101", "A", 0, 0),
    ("ST102", "A", 0, 1),
    ("ST103", "A", 0, 2),  # reserved seat
    ("ST104", "B", 1, 0),
    ("ST101", "B", 0, 0),  # duplicate student ID
    ("ST105", "Z", 0, 0),  # unknown room
]

assigned_ids = set()
for student_id, room, row, seat in assignments:
    if room not in classrooms:
        print("REJECTED", student_id, "- unknown room", room)
        continue
    if student_id in assigned_ids:
        print("REJECTED", student_id, "- duplicate assignment")
        continue
    seats = classrooms[room]
    if row < 0 or row >= len(seats) or seat < 0 or seat >= len(seats[row]):
        print("REJECTED", student_id, "- invalid seat position")
        continue
    if seats[row][seat] == "RESERVED":
        print("REJECTED", student_id, "- seat is reserved")
        continue
    if seats[row][seat] is not None:
        print("REJECTED", student_id, "- seat already occupied")
        continue
    seats[row][seat] = student_id
    assigned_ids.add(student_id)
    print("ALLOCATED", student_id, "to", room, row, seat)

print("\nCLASSROOM SUMMARY")
for room, rows in classrooms.items():
    flat = [seat for row in rows for seat in row]
    occupied = sum(seat is not None and seat != "RESERVED" for seat in flat)
    reserved = flat.count("RESERVED")
    vacant = flat.count(None)
    print(room, "total:", len(flat), "occupied:", occupied,
          "reserved:", reserved, "vacant:", vacant)
    print("Layout:", rows)
```

Sample output

```
ALLOCATED ST101 to A 0 0
ALLOCATED ST102 to A 0 1
REJECTED ST103 - seat is reserved
ALLOCATED ST104 to B 1 0
REJECTED ST101 - duplicate assignment
REJECTED ST105 - unknown room Z

CLASSROOM SUMMARY
A total: 6 occupied: 2 reserved: 1 vacant: 3
B total: 4 occupied: 1 reserved: 1 vacant: 2
```

Understanding the solution

The nested list models rows and seats. The program rejects unknown rooms, invalid positions, reserved/occupied seats, and duplicate student IDs before allocating a seat.

CASE STUDY NOTES

9. Food Waste Monitoring System

Problem Statement

This program checks daily meal counts, calculates the percentage of prepared meals left over, and identifies days when food wastage is above the accepted limit.

Objectives

- To validate the number of meals prepared, served, and left over each day.
- To calculate food wastage as a percentage of meals prepared.
- To compare each day's wastage percentage with the accepted limit.
- To report days that exceed the limit and flag inconsistent data.

Assumptions

- The number of meals prepared must be greater than zero.
- Served and leftover meal counts must not be negative.
- The sum of meals served and leftovers must not exceed the number prepared; inconsistent records are reported.
- Wastage percentage is calculated as (leftovers ÷ prepared) × 100.
- The example acceptable wastage limit is 10%.
- The daily meal counts and the limit are sample values for learning purposes.

These notes are written to match the Python code and sample data in the original workbook. Existing pages and code have been kept unchanged.

9. Food Waste Monitoring System

Problem statement

Compare prepared meals, meals served, and leftovers over several days, calculate wastage percentages, and identify days exceeding acceptable limits.

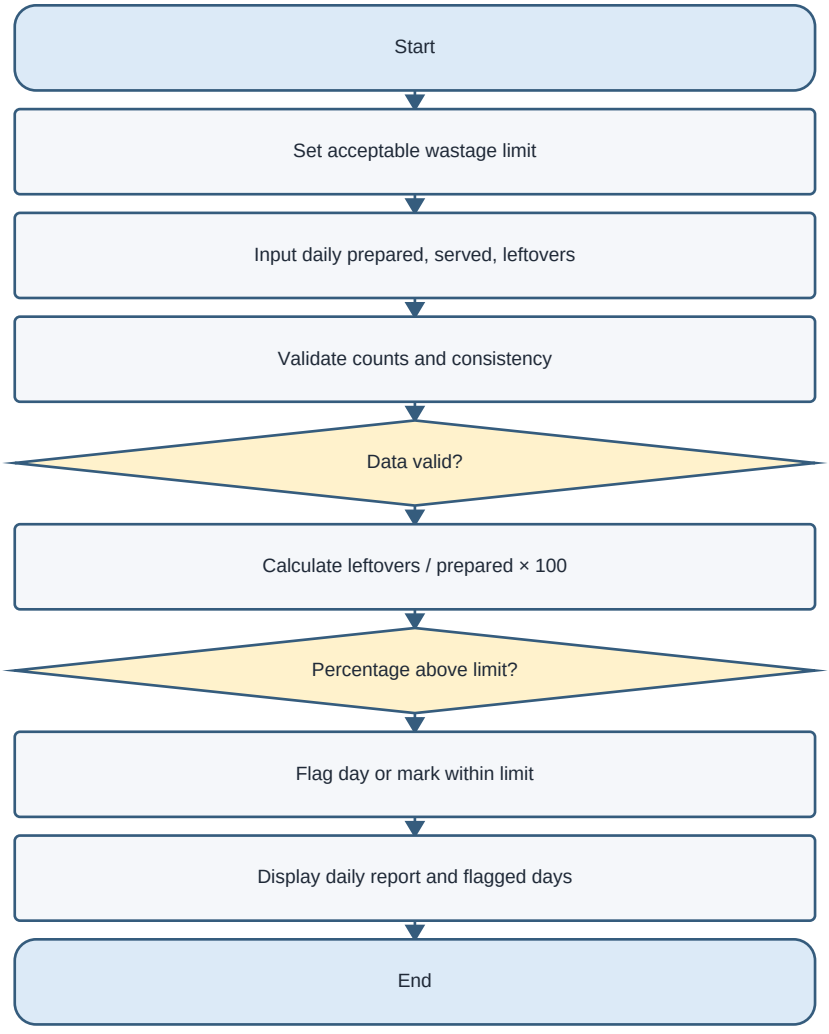
Assumptions and rules

• Prepared meals must be positive; served and leftover counts must be non-negative.

• Counts should satisfy served + leftovers <= prepared. Any mismatch is reported for review.

• Wastage percentage is leftovers / prepared × 100. The illustrative acceptable limit is 10%.

Flowchart



Pseudocode

```
BEGIN
  SET acceptable wastage percentage
  INPUT daily prepared, served, and leftover counts
  FOR each day
    VALIDATE counts and consistency
    IF invalid, report data error
    ELSE
      CALCULATE wastage percentage
      IF percentage exceeds limit, flag day
      ELSE mark day within limit
```

```
    ENDIF
  ENDFOR
  DISPLAY daily percentages and flagged days
END
```

9. Food Waste Monitoring System — Python Program

Python source code

```
# Food Waste Monitoring System
LIMIT_PERCENT = 10
daily_data = [
    ("Monday", 200, 170, 30),
    ("Tuesday", 180, 170, 10),
    ("Wednesday", 220, 190, 30),
    ("Thursday", 150, 145, 5),
    ("Friday", 200, 180, 25),  # inconsistent: 180 + 25 > 200
]

exceeded_days = []
for day, prepared, served, leftovers in daily_data:
    if prepared <= 0 or served < 0 or leftovers < 0:
        print(day, "- INVALID negative/zero counts")
        continue
    if served + leftovers > prepared:
        print(day, "- DATA ERROR: served + leftovers exceeds prepared")
        continue
    waste_percent = leftovers / prepared * 100
    status = "EXCEEDS LIMIT" if waste_percent > LIMIT_PERCENT else "Within limit"
    print(day, "waste:", round(waste_percent, 2), "% -", status)
    if waste_percent > LIMIT_PERCENT:
        exceeded_days.append(day)

print("Acceptable limit:", LIMIT_PERCENT, "%")
print("Days exceeding limit:", exceeded_days or "None")
```

Sample output

```
Monday waste: 15.0 % - EXCEEDS LIMIT
Tuesday waste: 5.56 % - Within limit
Wednesday waste: 13.64 % - EXCEEDS LIMIT
Thursday waste: 3.33 % - Within limit
Friday - DATA ERROR: served + leftovers exceeds prepared
Acceptable limit: 10 %
Days exceeding limit: ['Monday', 'Wednesday']
```

Understanding the solution

The program validates counts before calculating leftovers as a percentage of prepared meals. Inconsistent data is reported rather than used to produce a misleading waste percentage.

CASE STUDY NOTES

10. Campus Resource Allocation Under Constraints

Problem Statement

This program distributes available campus equipment among activities in priority order. It prevents allocation beyond available stock and reports any resources that are still needed.

Objectives

- To store the available quantity of each equipment type.
- To process activities in the given priority order.
- To allocate the smaller of the required quantity and the remaining stock.
- To calculate shortages and display the remaining resources after all activities.

Assumptions

- Activities are processed in the order listed; an earlier activity has higher priority.
- Equipment quantities are whole-number counts, and available stock cannot become negative.
- Partial allocation is allowed when the full requirement cannot be met.
- Any unmet requirement is recorded as a shortage.
- Negative requirements are treated as invalid and are not allocated.
- The program uses projector, laptop, and microphone quantities as sample data.

These notes are written to match the Python code and sample data in the original workbook. Existing pages and code have been kept unchanged.

10. Campus Resource Allocation Under Constraints

Problem statement

Compare equipment requirements with available resources across activities, calculate shortages, and create a feasible allocation using a specified priority order.

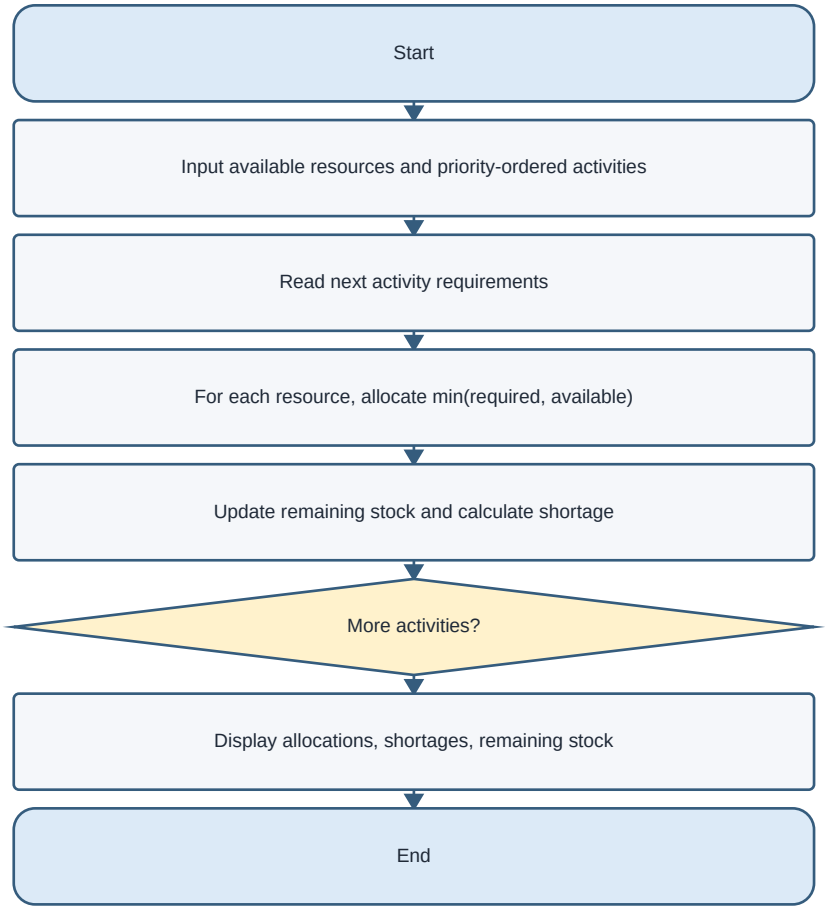
Assumptions and rules

• Activities are processed in the given priority order (highest priority first).

• Each activity lists required quantities for each equipment type. Allocation cannot exceed available stock.

• Partial allocation is allowed; unmet requirements are reported as shortages. A real scheduling or optimization problem may need additional constraints.

Flowchart



Pseudocode

```
BEGIN
  INPUT available stock for each resource
  INPUT activities in priority order and their requirements
  SET all allocated quantities to zero
  FOR each activity in priority order
    FOR each required resource
      ALLOCATE minimum of required amount and remaining stock
      REDUCE remaining stock by allocated amount
      CALCULATE shortage
```

```
    ENDFOR
    REPORT allocation and shortage for activity
  ENDFOR
  DISPLAY remaining stock and all allocation results
END
```

10. Campus Resource Allocation Under Constraints — Python Program

Python source code

```
# Campus Resource Allocation Under Constraints
available = {"projector": 3, "laptop": 10, "microphone": 2}
# Earlier item = higher priority
activities = [
    ("Exam", {"projector": 1, "laptop": 4, "microphone": 0}),
    ("AI Workshop", {"projector": 2, "laptop": 5, "microphone": 2}),
    ("Club Meeting", {"projector": 1, "laptop": 3, "microphone": 1}),
]

remaining = available.copy()
results = []
for activity, requirements in activities:
    allocated, shortages = {}, {}
    for resource, needed in requirements.items():
        if needed < 0:
            shortages[resource] = "invalid negative requirement"
            allocated[resource] = 0
            continue
        given = min(needed, remaining.get(resource, 0))
        allocated[resource] = given
        remaining[resource] = remaining.get(resource, 0) - given
        if given < needed:
            shortages[resource] = needed - given
    results.append((activity, allocated, shortages))
    print("\nActivity:", activity)
    print("Allocated:", allocated)
    print("Shortages:", shortages or "None")

print("\nREMAINING RESOURCES:", remaining)
print("Allocation policy: priority order; no resource is over-allocated.")
```

Sample output

```
Activity: Exam
Allocated: {'projector': 1, 'laptop': 4, 'microphone': 0}
Shortages: None

Activity: AI Workshop
Allocated: {'projector': 2, 'laptop': 5, 'microphone': 2}
Shortages: None

Activity: Club Meeting
Allocated: {'projector': 0, 'laptop': 1, 'microphone': 0}
Shortages: {'projector': 1, 'laptop': 2, 'microphone': 1}

REMAINING RESOURCES: {'projector': 0, 'laptop': 0, 'microphone': 0}
Allocation policy: priority order; no resource is over-allocated.
```

Understanding the solution

Activities are processed in priority order. For each resource, the program allocates the smaller of the required quantity and remaining stock, then records any shortage.